Rapport technique

APTISO : plateforme de mise en conformité ISO/IEC 27001 assistée par l'intelligence artificielle

Présenté par
Amina Bayoudh

Stage réalisé au sein de
ASM — All Soft Multimedia

Code source : `github.com/aminaoamina/APTISO`

Année universitaire 2025–2026

Sommaire

Introduction générale 1

1 Présentation du projet 2

1.1 Contexte : la mise en conformité ISO/IEC 27001 2
1.2 Objectifs de la plateforme 2
1.3 Périmètre fonctionnel 2
1.4 Exigences techniques 3

2 Architecture générale 4

2.1 Vue d'ensemble 4
2.2 Choix technologiques 4
2.3 Principes de conception 5
2.4 Organisation du dépôt 5

3 Conception des données 7

3.1 Base métier et Prisma 7
3.2 Diagramme de classes : noyau 7
3.3 Diagramme de classes : risques et Déclaration d'applicabilité 8
3.4 Diagramme de classes : amélioration, audit et revue 9
3.5 Diagramme de classes : IA et collaboration 10
3.6 Conventions, migrations et données de référence 11
3.7 Progression calculée 11

4 Backend NestJS 12

4.1 Organisation en modules 12
4.2 Cycle de traitement d'une requête 12
4.3 Règles métier appliquées par le serveur 13
4.4 Documents : génération, approbation et versions 13
4.5 Tâches planifiées et notifications 14
4.6 E-mails et récapitulatif hebdomadaire 14
4.7 Documentation de l'API (Swagger / OpenAPI) 15

5 Frontend Next.js 16

5.1 Structure des routes 16
5.2 Communication avec l'API 16
5.3 Gestion de l'état et composants clés 16

6 Service d'intelligence artificielle 18

6.1 Rôle et isolement 18
6.2 Modèles et registre de capacités 18
6.3 Sortie structurée et validation 19

6.4 Recherche documentaire hybride 19
6.5 Assistant ISO : contexte et flux 20
6.6 Traitements asynchrones 21
6.7 Base aptiso_ai et points d'entrée 22
6.8 Base de connaissances ISO 22
6.9 Évaluation de la qualité de l'IA 23

7 Sécurité 25
7.1 Authentification et sessions 25
7.2 Double authentification TOTP 25
7.3 Modèle d'autorisation 26
7.4 Protection des données et de l'API 26

8 Infrastructure et déploiement 28
8.1 Conteneurs Docker 28
8.2 Diagramme de déploiement 28
8.3 Configuration et secrets 29
8.4 Démarrage de l'environnement 29

9 Tests et qualité 30
9.1 Stratégie : tester avec les vrais composants 30
9.2 Suites de tests 30
9.3 Données de démonstration 30

10 Limites et perspectives 32
10.1 Limites 32
10.2 Perspectives 32

Conclusion générale 33

Bibliographie 34

Liste des figures

1.1 Les cinq phases du parcours APTISO et leur socle commun 3
2.1 Architecture générale d'APTISO 4
3.1 Diagramme de classes du noyau : utilisateurs, organisations, projets, étapes et documents 8
3.2 Diagramme de classes des registres d'étape : exigences, risques et SoA 9
3.3 Diagramme de classes de l'amélioration continue, de l'audit interne et de la revue de direction 10
3.4 Diagramme de classes des données d'IA et de collaboration 11
4.1 Cycle de traitement d'une requête dans le backend 12
4.2 Diagramme d'états d'un document 14
6.1 Chemin de validation d'une sortie structurée 19
6.2 Recherche hybride : proximité vectorielle et plein texte fusionnées (le classement par identifiants cités s'y ajoute) 20
6.3 Diagramme de séquence d'une question à l'assistant 21
6.4 Diagramme de séquence de la revue IA d'un document 22
7.1 Diagramme de séquence de la connexion avec double authentification 26
8.1 Diagramme de déploiement de l'environnement de développement 29

Liste des tableaux

1.1 Exigences non fonctionnelles et réponse technique 3
2.1 Technologies retenues et justification 5
2.2 Scripts du workspace racine 6
4.1 Modules du backend par domaine 12
4.2 Tâches planifiées du backend 14
5.1 Principales routes du frontend 16
6.1 Modèles par défaut 18
6.2 Registre des capacités (extrait) 19
6.3 Points d'entrée du service d'IA (préfixe /v1) 22
6.4 Résultats de l'évaluation, avant et après les évolutions 24
7.1 Niveaux de rôles et exemples de règles 26
8.1 Services Docker Compose 28
9.1 Suites de tests et contrôles de qualité 30

Introduction générale

La norme **ISO/IEC 27001:2022** définit les exigences d'un système de management de la sécurité de l'information (SMSI) [1]. Pour une organisation, l'obtenir demande des mois de travail : fixer un périmètre, évaluer et traiter les risques, choisir parmi les 93 mesures de l'Annexe A, rédiger des dizaines de documents, former le personnel, réaliser un audit interne puis une revue de direction. Ce travail est souvent mené avec des tableurs et des documents dispersés, sans vue d'ensemble ni traçabilité.

APTISO répond à ce besoin. C'est une plateforme web qui guide une organisation à travers les cinq phases d'un projet ISO 27001, sur le modèle de Conformio [3], et qui conserve chaque décision, chaque document et chaque preuve. Un assistant d'intelligence artificielle aide à remplir les assistants de rédaction, relit les documents, lit les preuves numérisées et répond aux questions sur la norme et sur le projet, toujours sous le contrôle d'une personne.

Ce rapport décrit la réalisation technique d'APTISO pendant le stage effectué au sein d'**ASM — All Soft Multimedia**. Il présente le projet et ses exigences (chapitre 1), l'architecture générale (chapitre 2), la conception des données avec les diagrammes de classes (chapitre 3), puis chaque composant : le backend (chapitre 4), le frontend (chapitre 5) et le service d'IA (chapitre 6). Il traite ensuite la sécurité (chapitre 7), l'infrastructure et le déploiement (chapitre 8), les tests (chapitre 9), et se termine par les limites et les perspectives (chapitre 10).

Chapitre 1

Présentation du projet

1.1 Contexte : la mise en conformité ISO/IEC 27001

Un projet ISO 27001 produit des **informations documentées** (politiques, procédures, registres, enregistrements) qui doivent être approuvées, diffusées, conservées et revues (clause 7.5). Il repose aussi sur des **décisions tracées** : acceptation des risques résiduels par leurs propriétaires, approbation du plan de traitement par la direction, conclusions de la revue de direction. L'auditeur de certification vérifie la cohérence de l'ensemble : une mesure déclarée applicable dans la Déclaration d'applicabilité (SoA) doit être reliée à un risque, à une politique et à des preuves.

Deux publics utilisent la plateforme : les **organisations** qui mettent en œuvre la norme (responsable de projet, direction, propriétaires d'actifs, équipes techniques) et les **auditeurs**, internes ou externes, qui consultent les registres et les documents en lecture seule.

1.2 Objectifs de la plateforme

- **Guider** : découper le projet en phases et en étapes, chacune avec son assistant de saisie, son registre ou son document, et une liste de contrôle de fin d'étape.
- **Faire respecter les règles de la norme** par le serveur : une étape ne se termine que lorsque ses conditions sont remplies et que son document est approuvé dans la bibliothèque.
- **Tracer** : versions de documents, approbations, journal d'audit des actions, journal des appels à l'IA et des décisions humaines.
- **Assister par l'IA sans lui déléguer la décision** : l'IA propose, une personne accepte ou rejette.
- **Faire vivre le SMSI** après la certification : tâches récurrentes, rappels, programme d'audit sur trois ans.

1.3 Périmètre fonctionnel

Le parcours compte cinq phases et 19 étapes de base ; la phase 3 reçoit en plus une étape par politique exigée par la SoA (21 politiques pour le projet de démonstration). La figure 1.1 résume ce que produit chaque phase.

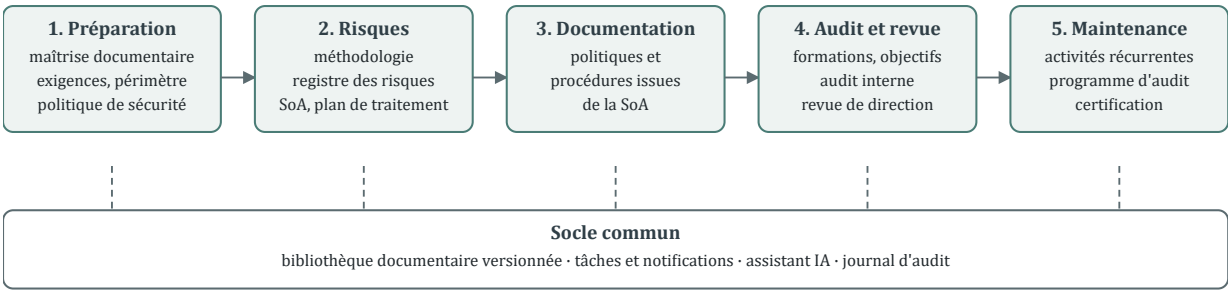


Figure 1.1 : Les cinq phases du parcours APTISO et leur socle commun.

1.4 Exigences techniques

Les exigences non fonctionnelles découlent de la nature du produit : un outil de sécurité doit lui-même être sûr, et un outil d'audit doit lui-même être traçable.

Tableau 1.1 : Exigences non fonctionnelles et réponse technique.

Exigence	Réponse technique
Traçabilité (clause 7.5)	versions de documents conservées avec leur PDF, approbateur et date ; journal d'audit ; journal `ai_runs` des appels IA
Confidentialité des données	droits relus en base à chaque requête, cookies HttpOnly, double authentification, secrets chiffrés (AES-256-GCM)
Exactitude de l'IA	réponses structurées validées par schéma et par des contrôles métier ; citations vérifiées ; décision humaine enregistrée
Disponibilité indépendante de l'IA	chaque fonction IA a un comportement de repli : une panne du fournisseur ne bloque aucun travail de conformité
Maintenabilité	TypeScript de bout en bout côté application, modules indépendants, client de base typé, migrations versionnées
Reproductibilité	versions de dépendances figées, infrastructure décrite dans Docker Compose, scripts de données de démonstration

Chapitre 2

Architecture générale

2.1 Vue d'ensemble

APTISO est composé de trois applications qui communiquent en HTTP : un **frontend** Next.js, un **backend** NestJS et un **service d'IA** FastAPI (figure 2.1). Le navigateur n'appelle que le backend ; le backend est le seul à lire la base métier et le seul à appeler le service d'IA, qui n'est pas exposé sur le réseau.

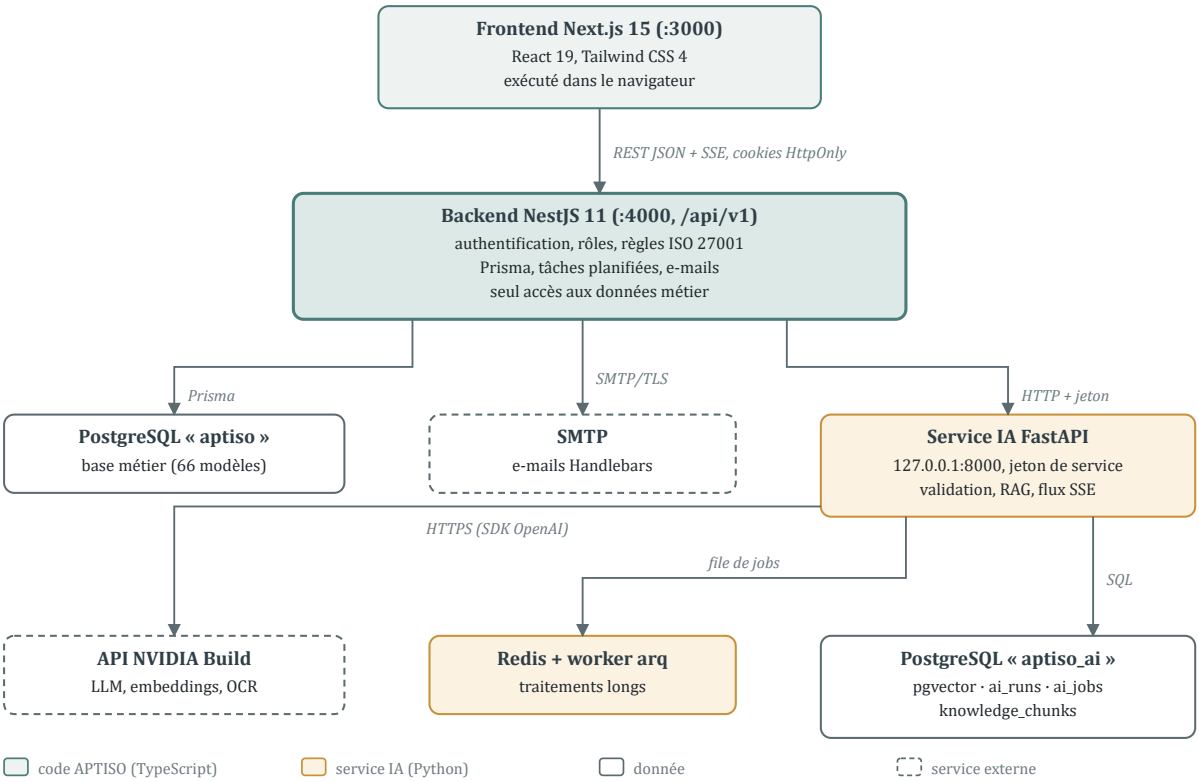


Figure 2.1 : Architecture générale d'APTISO.

2.2 Choix technologiques

Tableau 2.1 : Technologies retenues et justification.

Couche	Technologie	Justification
Interface	Next.js 15 (App Router), React 19, Tailwind CSS 4, Radix UI	routage par dossiers, composants accessibles, rendu rapide ; écosystème TypeScript partagé avec le backend [4]
API métier	NestJS 11, TypeScript	architecture modulaire, injection de dépendances, guards et pipes de validation, documentation Swagger générée [5]
Accès aux données	Prisma 6, PostgreSQL 16	schéma unique, client typé vérifié à la compilation, migrations SQL versionnées [6]
Service d'IA	Python 3.12, FastAPI, Pydantic, SQLAlchemy 2 (asyncpg), Alembic	écosystème IA en Python, validation par schéma, asynchrone de bout en bout [7, 14]
Recherche	pgvector 0.8 (HNSW, `halfvec`) + plein texte PostgreSQL	recherche hybride dans la base existante, sans moteur supplémentaire [8]
Traitements longs	Redis 7 + arq	file légère pour l'asyncio Python, relance des tâches [11]
Modèles	API NVIDIA Build via le SDK OpenAI	modèles ouverts récents (texte, embeddings, vision), interface standard interchangeable [10]
Édition, dessin	Tiptap (ProseMirror), tldraw 4	documents en JSON structuré exportables ; tableau blanc collaboratif [15, 16]
Infrastructure	Docker Compose	environnement reproductible : base, file, service d'IA et worker [17]

2.3 Principes de conception

- **Le serveur décide.** Le navigateur n'est jamais cru sur l'identité, le rôle ou l'état qu'il affirme ; toutes les règles de la norme sont appliquées dans les services du backend.
- **Séparation métier / IA.** Le métier (TypeScript) et l'IA (Python) évoluent indépendamment. Le service d'IA ne lit jamais la base métier : il ne reçoit que les données déjà filtrées par les droits de la personne.
- **« L'IA propose, le code vérifie, l'humain décide ».** Toute sortie de modèle exploitable par le programme est validée avant d'être utilisée ; toute proposition est acceptée ou rejetée par une personne.
- **Deux bases distinctes** sur le même serveur PostgreSQL : `aptiso` (métier, Prisma) et `aptiso_ai` (IA, Alembic), avec des rôles de connexion séparés.
- **Dégradation contrôlée.** Chaque fonction IA possède un repli sans IA ; les envois d'e-mails sont « au mieux » et ne bloquent jamais l'action qui les déclenche.

2.4 Organisation du dépôt

Le code vit dans un seul dépôt Git (monorepo), publié sur GitHub : `https://github.com/aminaoamina/APTISO` [19]. Les applications JavaScript et le paquet partagé sont

déclarés comme **workspaces npm** dans le `package.json` racine ; le service d'IA, en Python, est dans le même dépôt mais hors de npm.

```
APTISO/
├── apps/
│   ├── frontend/        @aptiso/frontend      Next.js 15 + React 19
│   ├── backend/         @aptiso/backend       NestJS 11 + Prisma 6 (schema.prisma, 49 migrations)
│   └── ai/              service IA            FastAPI, Dockerfile, migrations Alembic
├── packages/
│   └── shared-types/    @aptiso/shared-types  énumérations TypeScript communes
├── docker-compose.yml   PostgreSQL (pgvector), Redis, service IA + worker
└── package.json         "workspaces": ["apps/*", "packages/*"], scripts racine
```

Une seule commande `npm install` installe les dépendances de toutes les applications dans un `node_modules` commun (une dépendance partagée n'est installée qu'une fois) et un seul fichier de verrouillage garantit les mêmes versions sur toutes les machines.

Tableau 2.2 : Scripts du workspace racine.

Commande	Effet
`npm run dev`	lance ensemble le frontend (`next dev`, port 3000) et le backend (`nest start --watch`, port 4000) avec `concurrently`, journaux préfixés `fe` / `be`
`npm run build`	compile `shared-types`, puis chaque application
`npm run lint`, `npm run typecheck`	ESLint et `tsc --noEmit` dans chaque workspace
`npm run db:generate` | `db:migrate` | `db:studio`	commandes Prisma, déléguées au workspace backend
`npm run test:scenario -w @aptiso/backend`	scénario de bout en bout (chapitre 9)

Chapitre 3

Conception des données

3.1 Base métier et Prisma

La base métier `aptiso` (PostgreSQL 16) compte **66 modèles** et **50 énumérations**, décrits dans un fichier unique `apps/backend/prisma/schema.prisma`. Prisma génère à partir de ce schéma un client TypeScript typé : chaque requête est vérifiée à la compilation, si bien qu'une colonne renommée casse la compilation au lieu de casser l'exécution. Les sections suivantes présentent le modèle sous forme de quatre diagrammes de classes UML ; seuls les attributs significatifs sont montrés, les identifiants (UUID) et dates techniques sont omis. Le losange plein indique une composition : la suppression du conteneur supprime ses composants.

3.2 Diagramme de classes : noyau

L'organisation porte les projets ; un projet porte ses cinq phases, chaque phase ses étapes, et chaque étape au plus un document, lui-même composé de ses versions publiées. Les personnes accèdent aux organisations et aux projets par leurs appartenances, qui portent les rôles.

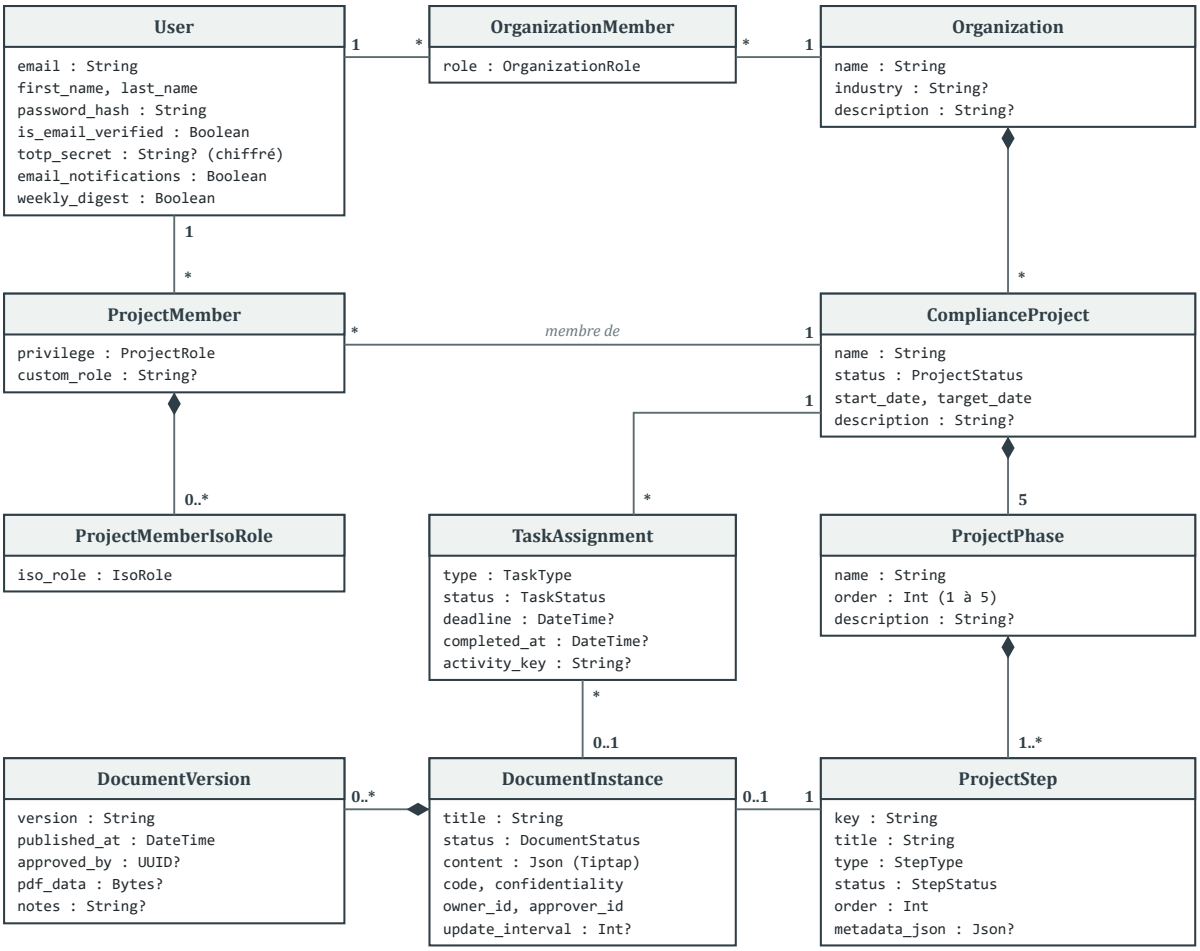


Figure 3.1 : Diagramme de classes du noyau : utilisateurs, organisations, projets, étapes et documents.

- **Trois niveaux de rôles** : `OrganizationMember.role` (propriétaire, administrateur, membre), `ProjectMember.privilege` (responsable, membre, auditeur) et `ProjectMemberIsoRole` (sept rôles ISO : direction, RSSI, propriétaire d'actif, chef de service, administrateur IT, auditeur interne, employé).
- **`DocumentInstance`** contient le texte courant (JSON Tiptap) et les données de maîtrise documentaire (code, confidentialité, propriétaire, approbateur, intervalle de revue) ; **`DocumentVersion`** fige chaque version approuvée avec son PDF.
- **`TaskAssignment`** relie une personne à un travail : tâche manuelle, approbation de document, ou tâche créée par un registre (action corrective, revue annuelle des risques…), identifiée par `activity_key` pour éviter les doublons.

3.3 Diagramme de classes : risques et Déclaration d'applicabilité

Les registres de la phase 2 sont rattachés à leur étape. Un risque est la combinaison d'un actif, d'une vulnérabilité et d'une menace ; il est évalué (impact et vraisemblance de 0 à 2, niveau égal à leur somme, acceptable jusqu'à 2), traité par des mesures de l'Annexe A (association n–n par la table `RiskItemControl`), puis approuvé par son propriétaire. La SoA contient exactement 93 lignes, une par mesure du catalogue.

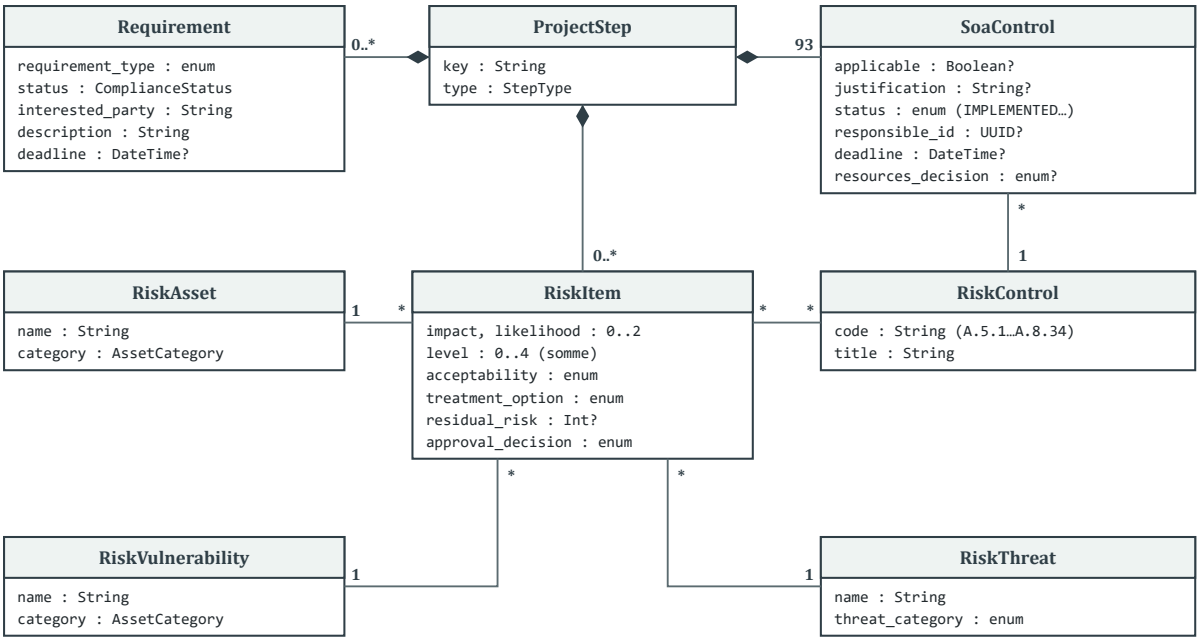


Figure 3.2 : Diagramme de classes des registres d'étape : exigences, risques et SoA.

Ce modèle porte une règle d'audit vérifiée par le serveur : une mesure utilisée dans le traitement d'un risque ne peut pas être déclarée non applicable dans la SoA. Le plan de traitement ne peut être confirmé que lorsque chaque mesure applicable a un statut, et l'approbation des propriétaires de risques exige que les demandes de ressources aient été tranchées par la direction.

3.4 Diagramme de classes : amélioration, audit et revue

Les registres de la phase 4 sont rattachés au projet. Un constat d'audit interne en non-conformité crée automatiquement une `Nonconformity` ; un incident de sécurité peut aussi en créer une. Chaque non-conformité porte ses actions correctives, et chaque décision de revue de direction son responsable et son échéance.

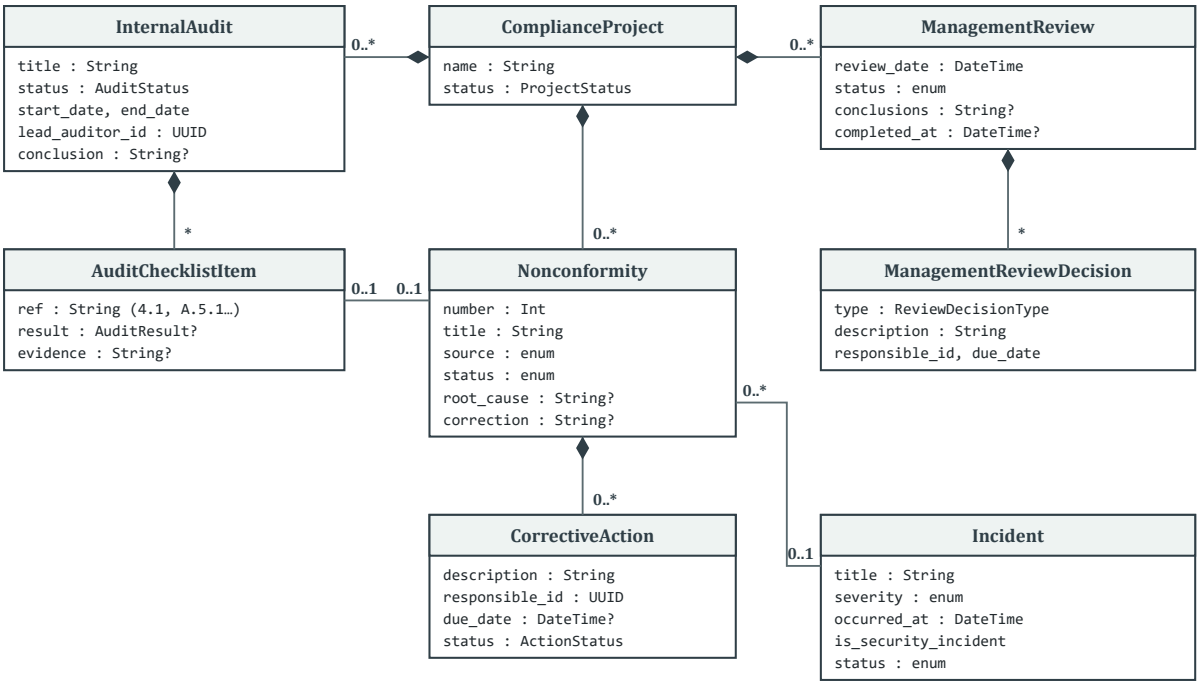


Figure 3.3 : Diagramme de classes de l'amélioration continue, de l'audit interne et de la revue de direction.

Les formations (`Training`), les objectifs de sécurité (`SecurityObjective` et leurs mesures) et les preuves (`Evidence`, avec fichiers, liens vers les enregistrements et date d'expiration) suivent le même schéma de rattachement au projet.

3.5 Diagramme de classes : IA et collaboration

La base métier conserve les données d'IA qui appartiennent au projet et sont soumises à ses droits : les conversations de l'assistant (privées à leur auteur), les revues de documents et leurs constats, que l'utilisateur corrige ou écarte. Le journal technique des appels est, lui, dans la base `aptiso_ai` (section 6.7).

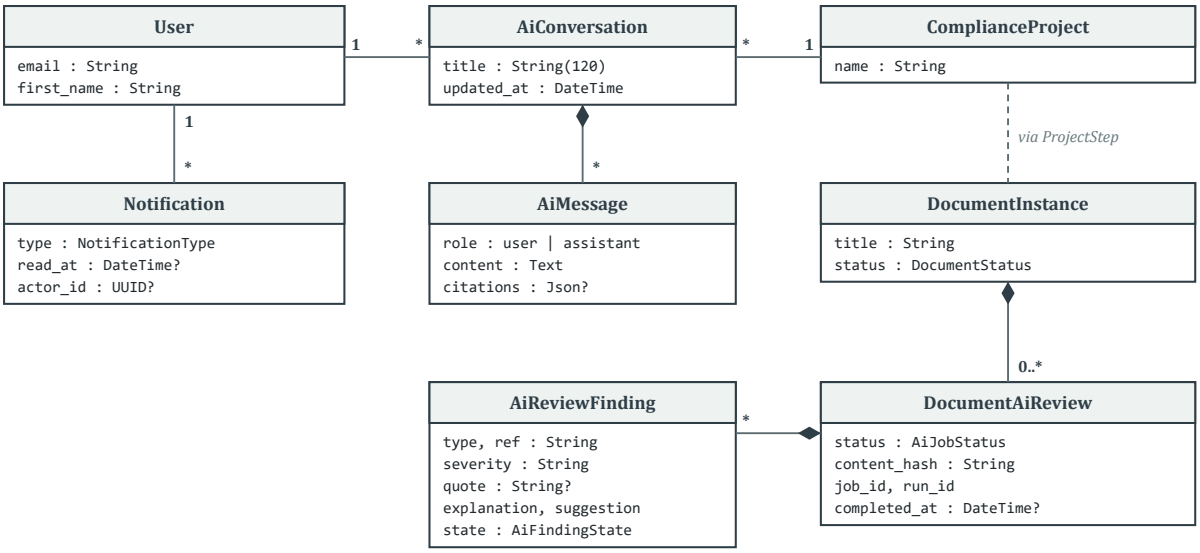


Figure 3.4 : Diagramme de classes des données d'IA et de collaboration.

3.6 Conventions, migrations et données de référence

- Identifiants **UUID**, dates en `timestamp(6)`, tables en `snake_case` (`@@map`), modèles en `PascalCase`.
- **Suppressions en cascade** déclarées sur les relations de composition ; `SetNull` pour les références facultatives (une étape supprimée laisse ses tâches orphelines plutôt que de les détruire).
- **Contraintes d'unicité porteuses de règles** : une phase par ordre et par projet, une étape par clé et par phase, un seul document par étape, un rôle ISO une seule fois par membre.
- **Index** sur les accès fréquents, par exemple `(project_id, user_id, updated_at)` pour l'historique de l'assistant.
- **Champs JSON** pour le contenu riche : texte Tiptap, réponses des assistants, citations, métadonnées d'étape.

Chaque évolution du schéma est une **migration SQL versionnée** (`prisma/migrations`, 49 à ce jour) appliquée par `prisma migrate deploy`. Après une migration écrite à la main, `prisma migrate diff` confirme qu'il n'y a aucun écart entre la base et le schéma. Les données de référence — catalogue des 25 exigences et 93 mesures de l'Annexe A (version `iso27001-2022@3`), modèles de documents et questions des assistants — sont insérées par migration ou au démarrage, avec un numéro de version.

3.7 Progression calculée

L'avancement n'est jamais stocké : il est **déduit des données**. Une étape est « en cours » dès qu'elle a un document, des enregistrements de registre, des tâches ou des demandes ; elle est « terminée » lorsqu'elle a été clôturée ; une phase suit ses étapes. Une fonction partagée (`project-progress.ts`) fait ce calcul pour la page du projet, la liste des projets et l'assistant IA, qui affichent donc toujours la même chose.

Chapitre 4

Backend NestJS

4.1 Organisation en modules

Le backend est une API REST NestJS 11 (port 4000, préfixe `/api/v1`, documentation Swagger sur `/api/docs`). Chaque domaine est un module : un contrôleur expose les routes, un service porte la logique, des DTO décrivent et valident les entrées.

Tableau 4.1 : Modules du backend par domaine.

Domaine	Modules
Comptes et accès	`auth` (inscription, connexion, sessions, 2FA), `organizations`, `team`, `notifications`, `mail`, `digest`
Projet	`projects` (phases, étapes, progression), `frameworks`, `tasks`, `requests` (demandes de ressources)
Phase 1	`documents` (assistants, bibliothèque, versions, export Word et PDF), `requirements`
Phase 2	`risk-register`, `soa`
Phase 3	`policies` (politiques générées depuis la SoA)
Phase 4	`audit-prep` (formations, objectifs, audit interne, revue de direction, non-conformités, incidents), `evidence`
Phase 5	`maintenance` (activités récurrentes, programme d'audit)
Transverses	`ai` (client du service d'IA, contexte, assistant, conversations), `exports` (Excel, ZIP), `connectors` (Google Drive), `whiteboard`, `prisma`

4.2 Cycle de traitement d'une requête

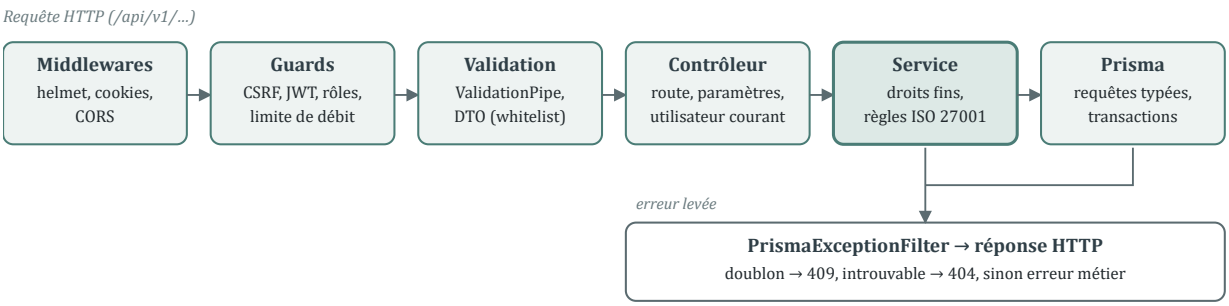


Figure 4.1 : Cycle de traitement d'une requête dans le backend.

1. **Middlewares globaux** (`main.ts`) : `helmet` pour les en-têtes de sécurité, `cookie-parser`, CORS limité à l'origine du frontend avec transmission des cookies.
2. **Guards** : `CsrfGuard` et `JwtAuthGuard` sont globaux (une route publique est marquée `@Public()`) ; selon la route, `OrganizationRoleGuard` ou `ProjectRoleGuard` avec `@ProjectRoles(…)`, et `ThrottlerGuard` sur les routes sensibles.
3. **ValidationPipe** global : les DTO décorés avec `class-validator` refusent tout champ inconnu (`whitelist`, `forbidNonWhitelisted`) et convertissent les types.
4. **Service** : `ProjectAccessService` relit en base le rôle du membre et calcule les droits fins (`canEdit`, `canDecide`), puis les règles métier s'appliquent avant l'écriture Prisma, souvent dans une transaction.
5. **Filtre d'exceptions** : `PrismaExceptionFilter` traduit les erreurs de base (doublon, enregistrement absent) en réponses HTTP claires.

4.3 Règles métier appliquées par le serveur

Les règles de la norme ne sont pas de simples indications à l'écran : elles sont vérifiées dans les services, avec un message d'erreur qui dit ce qui manque. Exemples :

- une étape obligatoire ne peut pas être ignorée ; une étape de registre ne se termine que lorsque sa liste de contrôle est complète **et** que son document généré est dans la bibliothèque ;
- le registre des exigences (clause 4.2) ne peut pas être terminé vide ;
- une mesure utilisée dans un traitement de risque ne peut pas être exclue de la SoA ; modifier le plan après son approbation réinitialise l'approbation des propriétaires ;
- seule la direction (rôle ISO « direction » ou responsable de projet) approuve les ressources, les formations, les objectifs et la revue de direction ;
- une formation ne passe à « réalisée » qu'avec une date et une preuve ; une non-conformité ne se clôt qu'avec correction, cause racine, actions terminées et revue d'efficacité ;
- un auditeur en lecture seule peut réaliser un audit interne qui lui est attribué, mais pas en modifier le paramétrage.

4.4 Documents : génération, approbation et versions

Les assistants de rédaction produisent un document à partir de modèles TypeScript (`documents/templates`) qui reçoivent les réponses du formulaire ; les politiques de la phase 3 sont générées section par section à partir des mesures applicables de la SoA. Le contenu est stocké en JSON Tiptap, ce qui permet l'édition riche, l'export Word (bibliothèque `docx`), le rendu PDF et l'indexation par l'IA. La figure 4.2 montre le cycle de vie d'un document.

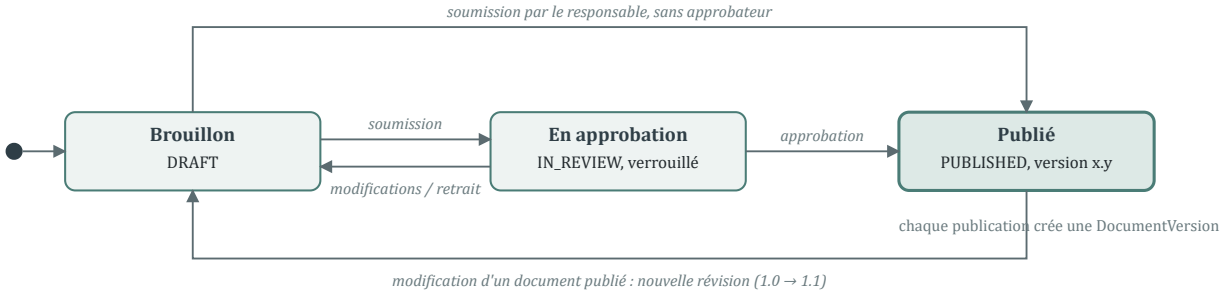


Figure 4.2 : Diagramme d'états d'un document.

Le bloc de maîtrise documentaire (code, version, propriétaire, approbateur, historique des modifications) n'est jamais figé dans le texte : il est imprimé à l'export à partir des données de la bibliothèque, si bien qu'il reste toujours exact (clause 7.5.2). Un document qui a des versions publiées ne peut pas être supprimé.

4.5 Tâches planifiées et notifications

Les tâches planifiées utilisent `@nestjs/schedule`. Elles sont idempotentes : exécutées deux fois, elles ne créent pas de doublon.

Tableau 4.2 : Tâches planifiées du backend.

Fréquence	Tâche	Effet
Chaque nuit, 3 h	Maintenance du SMSI	crée les tâches des activités récurrentes arrivées à échéance (revues des risques, incidents, objectifs et documents ; audit interne ; revue de direction)
Chaque jour, 7 h	Renouvellement des preuves	crée une tâche quand une preuve approche de son expiration
Chaque jour, 8 h	Rappels d'échéance	notifie une seule fois la personne dont la tâche arrive à échéance
Chaque heure	Miroir Google Drive	copie les nouvelles versions publiées dans le Drive connecté de l'organisation
Lundi, 8 h	Récapitulatif hebdomadaire	un e-mail par personne sur l'ensemble de ses projets

Chaque événement utile crée une `Notification` affichée dans la cloche de l'application. Les types qui demandent une action sont aussi envoyés par e-mail, sauf si la personne a désactivé les e-mails de notification ; personne n'est notifié de ses propres actions.

4.6 E-mails et récapitulatif hebdomadaire

Les e-mails sont envoyés par Nodemailer (SMTP, TLS) avec des gabarits Handlebars en CSS en ligne, lisibles dans les principales messageries. Le récapitulatif hebdomadaire illustre la répartition des rôles entre code et IA :

1. le backend calcule les faits des sept derniers jours pour chaque projet de la personne (tâches et étapes terminées par l'équipe et par elle, tâches ouvertes et en retard, phase en cours, avancement) et ses prochaines actions ;
2. le service d'IA rédige le titre, les résumés, les prochaines étapes, un conseil lié à la phase et un mot d'encouragement ; tout nombre cité doit figurer dans les faits, sinon la réponse est refusée ;
3. sans IA, le même e-mail part avec des phrases simples et un conseil fixe par phase.

4.7 Documentation de l'API (Swagger / OpenAPI)

La documentation de l'API est **générée à partir du code** au démarrage du backend, au format OpenAPI 3, et publiée par Swagger UI sur `/api/docs` [18]. Elle ne peut donc pas diverger des routes réelles.

- **Source** : le module `@nestjs/swagger` lit les contrôleurs et leurs décorateurs : `@ApiTags` regroupe les routes par domaine (23 groupes : Authentication, Projects, Risk Register, Statement of Applicability, AI, Weekly digest…), `@ApiOperation` décrit chaque route, `@ApiResponse` ses codes de retour, et `@ApiProperty` sur les DTO décrit chaque champ avec son type, ses contraintes et un exemple.
- **Cohérence avec la validation** : les mêmes classes DTO servent à la documentation et à la validation (`class-validator`) ; un champ documenté est un champ vérifié.
- **Authentification** : le document déclare l'authentification par jeton (`addBearerAuth`) et les routes protégées portent `@ApiBearerAuth` ; l'option `persistAuthorization` conserve le jeton entre deux rechargements de la page de test.
- **Usage** : l'interface permet d'essayer chaque route directement depuis le navigateur ; le fichier OpenAPI peut aussi servir à générer un client ou à alimenter des tests de contrat.

Le service d'IA, écrit avec FastAPI, expose de la même façon sa propre documentation OpenAPI, générée à partir des modèles Pydantic de ses requêtes et réponses ; elle n'est accessible que depuis la machine, comme le service.

Chapitre 5

Frontend Next.js

5.1 Structure des routes

Le frontend est une application Next.js 15 (App Router) en React 19 et TypeScript, stylisée avec Tailwind CSS 4 et des composants Radix UI. Il ne contient aucune règle métier ni aucun secret. Le routage suit l'arborescence du dossier `app/` ; les segments dynamiques portent les identifiants et des gabarits `layout.tsx` imbriqués fournissent la barre latérale, l'en-tête et, dans un projet, l'assistant IA.

Tableau 5.1 : Principales routes du frontend.

Route	Rôle
`/login`, `/register`, `/verify-email`, `/forgot-password`, `/reset-password`, `/accept-invitation`	parcours d'authentification (pages publiques)
`/dashboard`, `/dashboard/tasks`, `/dashboard/profile`	espace personnel : tableau de bord, tâches, profil (2FA, préférences de notification)
`/dashboard/organizations/[orgId]` (+ `members`, `library`)	organisation : membres, invitations, bibliothèque documentaire, intégration Google Drive
`…/projects/[projectId]`	projet : phases et étapes (`steps/[stepId]`, assistants), `documents`, `registers`, `audit`, `requests`, `maintenance`, `team`, `members`, `whiteboard`

5.2 Communication avec l'API

- Un client **axios** unique (`lib/api-client.ts`) envoie toutes les requêtes avec `withCredentials` : les jetons voyagent dans des cookies HttpOnly que le JavaScript de la page ne voit jamais.
- **Rafraîchissement automatique** : sur une réponse 401, un intercepteur appelle `/auth/refresh` une seule fois, met en file les requêtes en attente, puis les rejoue.
- Un **jeton CSRF** est ajouté dans l'en-tête `x-csrf-token` des requêtes publiques qui modifient l'état.
- Les appels sont regroupés par domaine (`lib/api.ts`, `lib/ai-api.ts`) avec leurs types TypeScript.
- Les **réponses en flux** de l'IA (Server-Sent Events) sont lues au fil de l'eau grâce à `onDownloadProgress` d'axios, qui expose le texte reçu pendant le téléchargement ; un petit lecteur découpe les événements `delta`, `citations`, `done` et `error`.

5.3 Gestion de l'état et composants clés

L'état global est réduit au minimum : un store **Zustand** garde l'utilisateur connecté (persisté sans les jetons). Le reste est de l'état local rechargé depuis l'API ; le hook `usePollWhile` interroge le serveur tant qu'un traitement IA est en cours.

- **Éditeur de documents** basé sur Tiptap (ProseMirror) : sections, tableaux, amélioration d'un paragraphe par l'IA (plus clair, plus formel, plus court).
- **Assistant ISO** (`components/ai/companion-chat.tsx`) : réponses en flux, sources cliquables, historique des conversations, prochaines actions en accueil.
- **Registres interactifs** : registre des risques par étapes (actifs, vulnérabilités, menaces, évaluation, traitement), SoA, audit interne, revue de direction.
- **Tableau blanc** du projet avec tldraw 4, sauvegardé en instantané JSON.

Chapitre 6

Service d'intelligence artificielle

6.1 Rôle et isolement

Toute l'IA est isolée dans un service Python FastAPI (`apps/ai`, port 8000). Il n'écoute que sur l'interface locale, exige un jeton de service (`X-Service-Token`) et ne lit jamais la base métier. Le backend, qui connaît les droits de la personne, construit pour chaque appel un contexte limité à ce qu'elle peut voir ; les contenus fournis par les utilisateurs y sont balisés comme données non fiables pour se prémunir contre l'injection d'instructions.

6.2 Modèles et registre de capacités

Les modèles sont appelés par l'API NVIDIA Build avec le SDK OpenAI ; chacun est configurable par variable d'environnement.

Tableau 6.1 : Modèles par défaut.

Rôle	Modèle
Principal (texte, raisonnement)	`nvidia/nemotron-3-super-120b-a12b`
Secours	`openai/gpt-oss-20b`
Embeddings	`nvidia/nemotron-3-embed-1b` (2048 dimensions)
Lecture de scans (OCR)	`nemotron-3-nano-omni-30b-a3b-reasoning`, secours `nemotron-3.5-super-vl-120b-a12b`
Contre-lecture visuelle	`meta/llama-3.2-11b-vision-instruct`

Une fonctionnalité ne nomme jamais un modèle mais une **capacité**. Chaque capacité fixe les modèles essayés dans l'ordre, la température, la limite de jetons, le délai, les nouvelles tentatives (sur erreurs transitoires seulement), le mode JSON et l'activation du raisonnement, coûteux en latence, réservé aux tâches qui en ont besoin.

Tableau 6.2 : Registre des capacités (extrait).

Capacité	Température	Jetons max	Délai	Raisonnement	Usage
`wizard_suggest`	0,2	3 000	40 s	oui	suggestion de réponse d'assistant, JSON
`paragraph_improve`	0,3	1 200	30 s	non	réécriture d'un passage, en flux
`companion_chat`	0,3	1 500	45 s	non	assistant ISO, en flux
`document_review`	0,1	4 000	90 s	oui	revue d'un document, JSON
`evidence_analyze`	0,1	1 500	90 s	non	analyse d'une preuve, JSON
`next_actions`	0,2	800	25 s	non	classement des prochaines actions
`weekly_digest`	0,5	1 500	45 s	non	rédaction du récapitulatif

6.3 Sortie structurée et validation

Toute réponse que le programme exploite suit le même chemin (figure 6.1). Le modèle doit rendre un JSON, qui est réparé si besoin, validé par un schéma Pydantic, puis soumis à des **contrôles métier** propres à chaque fonction : identifiants cités réellement fournis, citations mot pour mot présentes dans les données, aucun nom de produit ou de lieu absent des faits, aucun chiffre inventé, aucun numéro de mesure de l'ancienne version 2013. En cas d'échec, une seule nouvelle tentative est faite avec la liste exacte des erreurs ; au second échec, la réponse est rejetée et le backend applique son comportement sans IA. Les invites sont du code versionné (`wizard-suggest@4`, `companion-chat@2`…) et leur version est journalisée avec chaque appel.

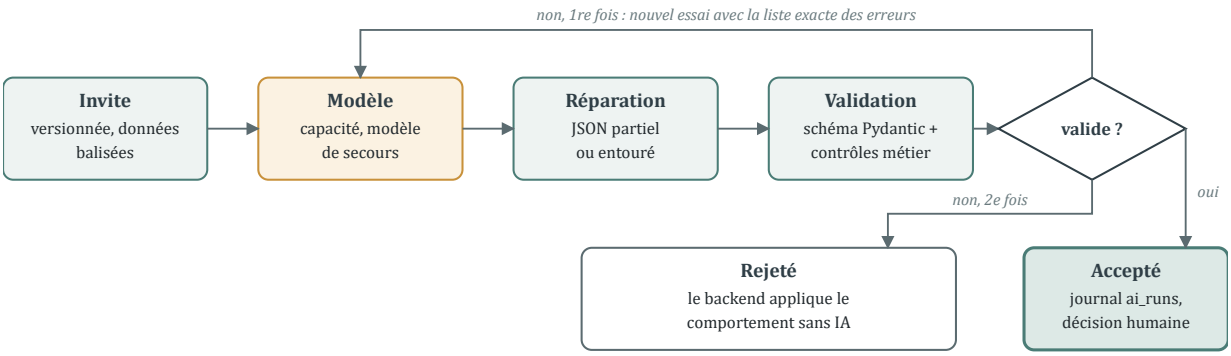


Figure 6.1 : Chemin de validation d'une sortie structurée.

6.4 Recherche documentaire hybride

Les documents publiés et les preuves analysées sont découpés en passages d'environ 1 500 caractères, coupés entre paragraphes, et stockés dans `knowledge_chunks` avec un vecteur `halfvec(2048)` (index HNSW, distance cosinus) et une colonne plein texte `tsvector`. La recherche produit trois classements — par sens (vecteurs), par mots (plein texte) et par identifiants de clauses ou de mesures cités dans la question (par exemple « A.8.13 ») — combinés par **Reciprocal Rank Fusion** [9] :

$$\text{score}(d) = \Sigma_{r \in \{\text{vecteur, texte, identifiants}\}} \, 1 / (k + \text{rang}_r(d) + 1), \text{ avec } k = 60$$

Les résultats sont filtrés par projet et par visibilité : un auditeur ne voit que les documents de la bibliothèque, un éditeur voit aussi les brouillons. Pour une question de suivi (« et à quelle fréquence ? »), la question précédente est ajoutée à la requête.

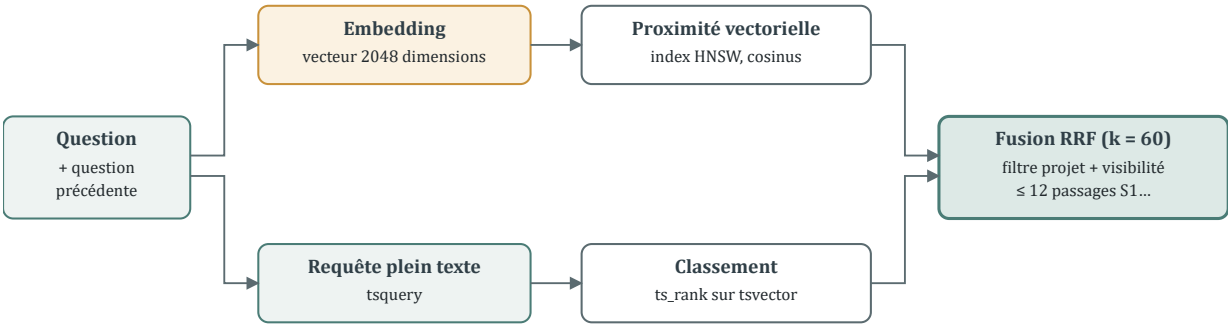


Figure 6.2 : Recherche hybride : proximité vectorielle et plein texte fusionnées (le classement par identifiants cités s'y ajoute).

6.5 Assistant ISO : contexte et flux

Pour chaque question, le backend vérifie les droits, ouvre ou reprend la conversation et charge ses six derniers messages depuis la base (jamais depuis le navigateur). Il obtient du service d'IA les passages pertinents, puis ajoute des **faits** tirés des registres : la progression du projet (phases, étapes non terminées, date de certification), et pour les clauses et mesures citées, les lignes de SoA, les risques et les documents correspondants. La réponse est diffusée en flux et enregistrée une fois terminée (figure 6.3). Seules les citations qui renvoient à un passage ou à un fait réellement fourni sont conservées.

Récupération structurée par parcours des relations. Les passages de texte ne suffisent pas pour une question comme « sommes-nous prêts pour A.8.13 ? » : la réponse dépend d'enregistrements liés entre eux. À partir des identifiants de clauses et de mesures renvoyés par la recherche, le backend parcourt donc les relations du modèle de données — mesure → ligne de SoA (applicabilité, statut, responsable, échéance) → risques qu'elle traite (propriétaire, niveau) → documents qui la couvrent (version, approbateur) → preuves associées — et fournit ces enregistrements à l'assistant comme **faits** numérotés ([F1], [F2]…), à côté des passages ([S1]…). Il s'agit d'une récupération de type graphe, légère : le graphe est le schéma relationnel lui-même, parcouru avec Prisma, sans base de graphe dédiée ni graphe construit par l'IA (GraphRAG). Les faits sont toujours filtrés par les droits de la personne (un auditeur ne voit que les documents publiés).

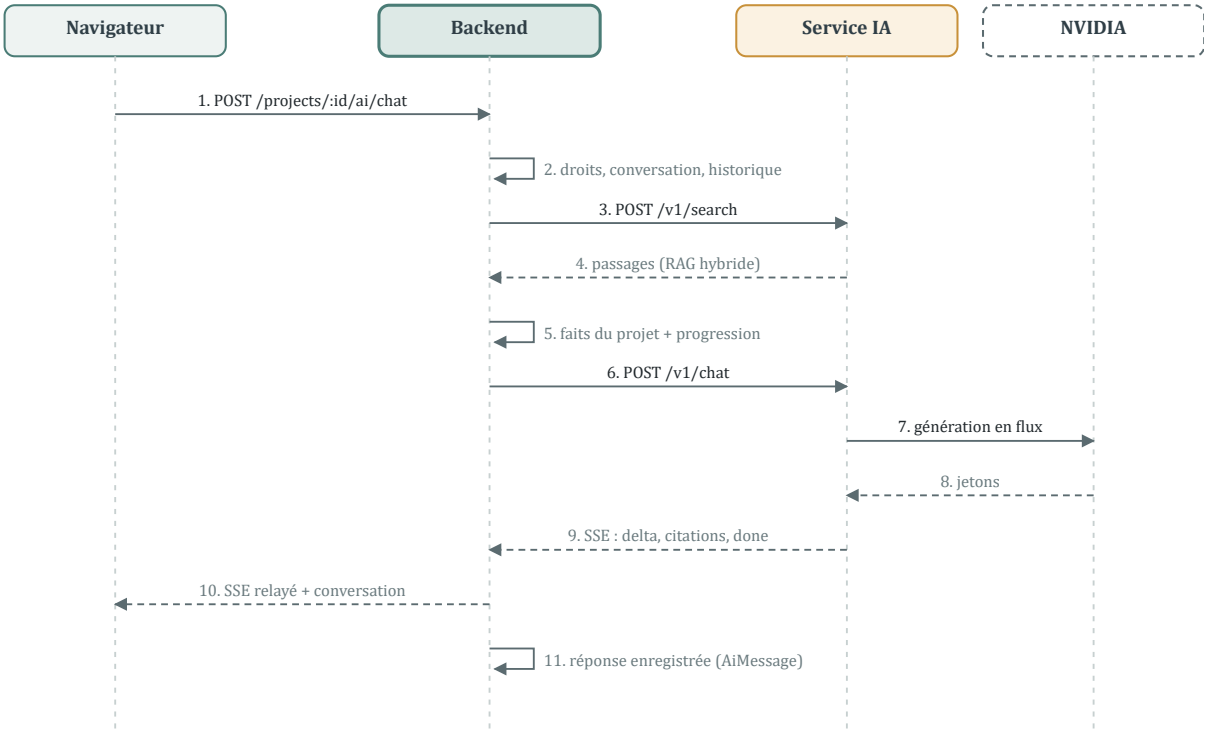


Figure 6.3 : Diagramme de séquence d'une question à l'assistant.

6.6 Traitements asynchrones

La revue d'un document et la lecture d'une preuve (couche texte avec PyMuPDF, sinon OCR avec contre-lecture par un second modèle) peuvent prendre plusieurs minutes. Elles passent par une file durable : la tâche est enregistrée dans `ai_jobs` avec une **clé d'idempotence** (empreinte du contenu et version de l'invite), si bien que deux demandes identiques renvoient la même tâche. Le worker arq l'exécute avec un bail et au plus trois tentatives ; un balayage périodique relance les tâches dont le bail a expiré.

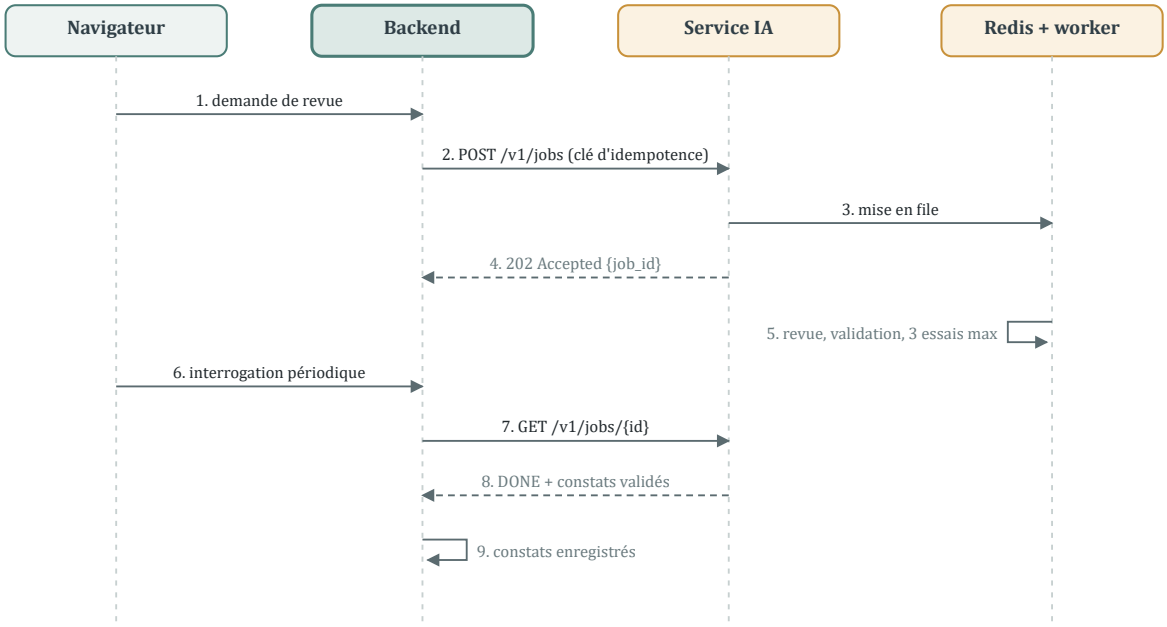


Figure 6.4 : Diagramme de séquence de la revue IA d'un document.

6.7 Base `aptiso_ai` et points d'entrée

La base du service d'IA (SQLAlchemy 2 asynchrone, migrations Alembic) contient trois tables : `ai_runs` (journal de chaque appel : capacité, modèle, version de l'invite, durée, jetons, résultat et décision humaine), `ai_jobs` (file durable) et `knowledge_chunks` (index de recherche).

Tableau 6.3 : Points d'entrée du service d'IA (préfixe /v1).

Route	Rôle
`POST /wizard/suggest`	suggestion de réponse pour une question d'assistant, avec ses sources
`POST /improve`	réécriture d'un paragraphe (flux SSE)
`POST /next-actions`, `POST /weekly-digest`	classement des actions, rédaction du récapitulatif
`POST /jobs`, `GET /jobs/{id}`	création (202) et suivi des traitements longs
`PUT|DELETE /index/documents/{id}`	indexation et suppression d'un document dans la recherche
`POST /search`, `POST /chat`	recherche hybride ; réponse de l'assistant (flux SSE)
`POST /runs/{id}/outcome`	enregistrement de la décision humaine sur une proposition
`GET /health/live`, `GET /health/ready`	santé du service, de la base, de pgvector et de Redis
`GET /metrics`	statistiques d'usage réel par capacité et versions des invites en service (section 6.9)

6.8 Base de connaissances ISO

Pour enseigner la norme et pas seulement la citer, l'assistant dispose d'une **base de connaissances propre à APTISO** : un guide de mise en œuvre pour chacune des 25 exigences des clauses 4 à 10 et des

93 mesures de l'Annexe A, soit **118 guides**. Chaque guide contient :

- l'**objectif** de l'exigence et le risque qu'elle traite ;
- **ce que vérifie l'auditeur** (3 à 5 points) ;
- des **exemples de preuves** (enregistrements, documents, journaux) ;
- les **erreurs fréquentes** relevées en audit.

Production. Les guides ont été rédigés une seule fois par le modèle principal, par lots de six, avec une invite qui impose de reformuler et interdit de reproduire le texte de la norme. Chaque lot est validé (schéma Pydantic, nombre d'éléments, longueur, refus de toute numérotation de 2013) et refait une fois avec la liste exacte des erreurs ; un lot refusé deux fois est repris exigence par exigence. Le résultat est un fichier versionné (`catalog/guides.json`, version `guides@1`) relu et intégré au dépôt : aucun guide n'est généré pendant l'utilisation.

Indexation. Les guides sont ajoutés au catalogue (titres exacts et questions d'audit) dans `knowledge_chunks` comme connaissance **partagée** entre tous les projets (`project_id` nul, type `iso`). La version de cette connaissance combine celle du catalogue et celle des guides (`iso27001-2022@3+g1`) : une nouvelle version déclenche automatiquement la réindexation. La recherche hybride retourne ainsi, pour une question, à la fois les passages des documents du projet et le guide de l'exigence concernée.

Utilisation. L'invite de l'assistant (`companion-chat@3`) présente ces guides comme une explication générale, distincte des faits de l'organisation : l'assistant les cite lorsqu'il enseigne (ce que vérifie l'auditeur, les preuves attendues, les erreurs à éviter), puis relie l'explication aux documents et registres du projet. Le texte officiel de la norme, protégé par le droit d'auteur et diffusé sous licence nominative, n'est pas intégré ; le même mécanisme d'indexation permettrait d'ajouter la copie d'une organisation qui détient sa propre licence, visible de ses seuls projets.

6.9 Évaluation de la qualité de l'IA

La qualité de l'IA est mesurée de deux façons complémentaires : un **jeu de cas annotés** exécuté sur la vraie plateforme, et des **statistiques d'usage réel** tirées du journal `ai_runs`.

Jeu de cas annotés

Le script `test/ai-eval.js` exécute 28 cas sur le projet de démonstration NovaTeck et les note **sans aucun modèle** (pas d'« IA qui juge l'IA »), par des règles déterministes :

- **Assistant (24 cas)** : 15 questions sur les faits du projet, avec les éléments que la réponse doit contenir (« Sarra Mansouri », « 2026-12-15 »…) ; 3 pièges dont la réponse n'existe pas dans les données (chiffre d'affaires, fournisseur du pare-feu, produit SIEM), où toute réponse inventée est comptée comme hallucination ; 6 questions d'apprentissage de la norme, dont un piège avec un numéro de mesure de 2013. Chaque question est posée par l'API réelle, comme le ferait l'utilisateur, puis la conversation est supprimée.
- **Revue de documents (4 cas)** : trois documents contenant des **défauts volontairement introduits** (formulation invérifiable, exigence manquante, contradiction avec la SoA) et un document correct, qui ne doit recevoir aucun constat grave.

Les indicateurs sont le taux de cas réussis, le rappel des faits attendus, la présence d'une source valide, l'exactitude des refus, le nombre de réponses contenant un fait inventé, le rappel et la précision des constats de revue, les fausses alertes et la latence. Chaque exécution enregistre ses résultats

(`test/eval/results`) avec les versions des invites mesurées ; un mode de re-notation recalcule les scores d'exécutions passées lorsqu'un cas est corrigé, pour que les comparaisons restent justes.

Tableau 6.4 : Résultats de l'évaluation, avant et après les évolutions.

Indicateur	Référence (invite v2)	Guides + invite v3	+ registres
Assistant : cas réussis	23 / 24	23 / 24	23 / 24
Assistant : rappel des faits attendus	95 %	95 %	**100 %**
Réponses avec une source valide (si exigée)	100 %	100 %	94 %
Refus corrects (information absente)	100 %	100 %	100 %
Réponses contenant un fait inventé	0	0	0
Latence médiane / 95e centile	2,9 s / 10,4 s	6,3 s / 14,2 s	4,6 s / 12,1 s
Revue : défauts introduits trouvés (rappel)	100 % (8/8)	100 % (8/8)	—
Revue : constats correspondant à un défaut (précision)	73 %	87 %	—
Revue : fausses alertes graves (document correct)	0	0	—

Lecture. Sur les questions de faits, l'assistant ne fabrique aucune information et refuse correctement lorsque la donnée n'existe pas. Les guides n'ont pas changé le score des questions de faits, ce qui est attendu, mais ils ont transformé les réponses d'apprentissage : pour la mesure A.8.13, l'assistant cite désormais le guide et précise ce que l'auditeur vérifie et les erreurs fréquentes (« sauvegardes jamais testées en restauration »). La précision de la revue est passée de 73 % à 87 %, et les constats non comptés sont pour la plupart justifiés (par exemple un chiffrement des sauvegardes exigé par la SoA mais absent de la politique). L'évaluation a enfin révélé une **lacune réelle** : l'assistant ne voyait pas les registres d'incidents et de non-conformités, qui ne sont pas des documents. Ils lui sont désormais fournis comme faits du projet : le rappel des faits atteint 100 % (dernière colonne). Le seul cas encore en échec contient les bons faits mais cite sa source au mauvais format (« (F7) » au lieu de « [F7] ») : la citation n'est donc pas reconnue. Le coût de ces contextes plus riches est une latence plus élevée.

Statistiques d'usage réel

Le point d'entrée `GET /v1/metrics` du service d'IA agrège le journal `ai_runs` par capacité, sur une période choisie : nombre d'appels, taux de réponses valides au premier essai et de réponses rejetées après validation, recours au modèle de secours, latence moyenne et 95e centile, jetons consommés, et surtout **taux d'acceptation des propositions par les utilisateurs** (décisions acceptées ou écartées). Sur les données de développement, par exemple, 57 % des suggestions des assistants de rédaction ont été acceptées et 4,3 % des réponses ont été rejetées par la validation. Ces chiffres complètent le jeu de cas : l'un mesure la qualité sur des cas connus, l'autre la confiance des utilisateurs en conditions réelles.

Chapitre 7

Sécurité

7.1 Authentification et sessions

- Mots de passe hachés avec **Argon2** ; un compte n'est actif qu'après vérification de l'adresse e-mail.
- Deux **JWT** : un jeton d'accès de 15 minutes et un jeton de rafraîchissement de 7 jours, chacun dans un **cookie HttpOnly** (`Secure` et `SameSite=Strict` en production) ; le cookie de rafraîchissement n'est envoyé qu'aux routes `/api/v1/auth`.
- **Sessions** enregistrées en base (appareil, adresse IP, dernière activité) : la personne les consulte et peut en révoquer une ou toutes.
- **Protection CSRF** par un guard global et un jeton dans l'en-tête des requêtes publiques qui modifient l'état.

7.2 Double authentification TOTP

La double authentification est facultative et proposée par une notification à chaque nouveau compte. Elle suit la RFC 6238 [12] (SHA-1, 6 chiffres, pas de 30 secondes, tolérance d'un pas), compatible avec Microsoft et Google Authenticator. Le secret est chiffré en base (AES-256-GCM) ; les dix codes de secours ne sont conservés que sous forme d'empreinte SHA-256 et servent une seule fois. Le dernier pas utilisé est mémorisé, ce qui empêche de rejouer un code pendant sa période de validité.

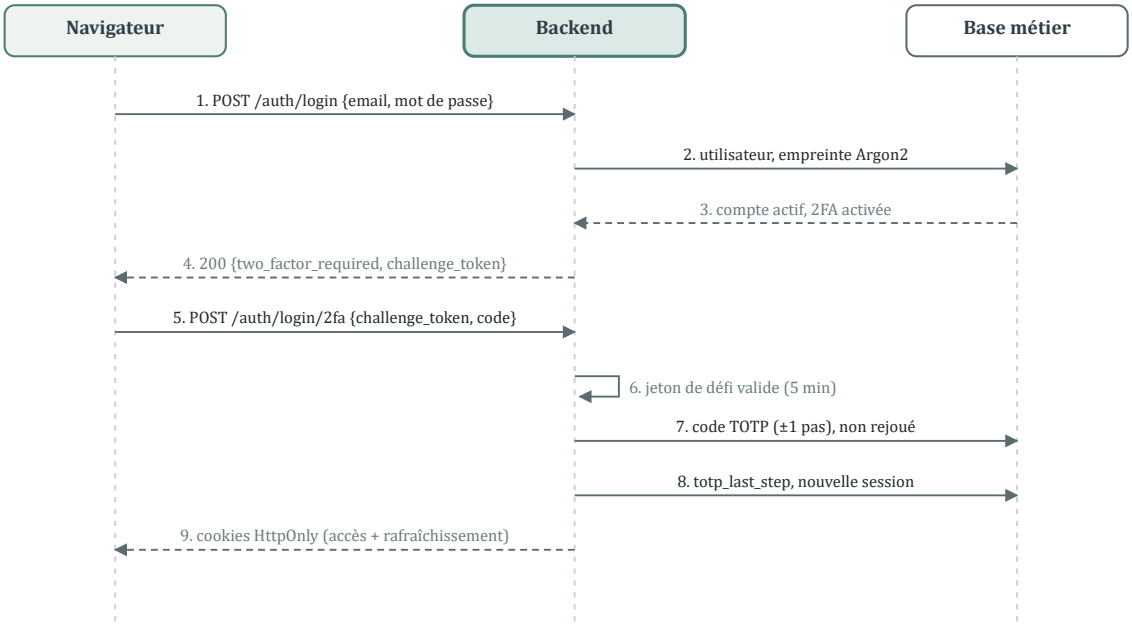


Figure 7.1 : Diagramme de séquence de la connexion avec double authentification.

Après le bon mot de passe, le serveur ne pose aucun cookie : il rend un **jeton de défi** de 5 minutes, dont l'unique usage est la seconde étape. Les cookies de session ne sont posés qu'après un code valide.

7.3 Modèle d'autorisation

Tableau 7.1 : Niveaux de rôles et exemples de règles.

Niveau	Rôles	Exemple de règle
Organisation	propriétaire, administrateur, membre	seuls propriétaire et administrateurs gèrent les membres et l'intégration Google Drive
Projet	responsable, membre, auditeur	l'auditeur est en lecture seule ; le responsable gère l'équipe, les tâches et la clôture des étapes
Rôles ISO	direction, RSSI, propriétaire d'actif, chef de service, administrateur IT, auditeur interne, employé	seule la direction (ou le responsable) approuve ressources, formations, objectifs et revue de direction

Les données personnelles restent privées : la conversation d'un membre avec l'assistant est invisible pour les autres, y compris le responsable du projet (réponse 404).

7.4 Protection des données et de l'API

- **Limitation de débit** : 10 tentatives de connexion par minute, 5 par minute pour les codes 2FA et l'envoi manuel du récapitulatif.
- **Journal d'audit** de toutes les actions sensibles : connexion, approbation, modification de registre, avec utilisateur, date et adresse IP.
- **Secrets chiffrés** en base (AES-256-GCM) : secret TOTP et jetons OAuth Google Drive, avec la clé `CONNECTOR_SECRET_KEY`.

- **Service d'IA non exposé** (127.0.0.1) et protégé par jeton ; contenus utilisateur balisés comme données non fiables dans les invites.
- **Validation stricte des entrées** (DTO, schémas Pydantic) et en-têtes de sécurité HTTP (`helmet`), conformément aux recommandations de l'OWASP [13].

Chapitre 8

Infrastructure et déploiement

8.1 Conteneurs Docker

Les services d'infrastructure sont décrits dans un unique `docker-compose.yml`. En développement, le frontend et le backend tournent directement sur la machine avec rechargement à chaud.

Tableau 8.1 : Services Docker Compose.

Service	Image	Port	Rôle
`postgres`	`pgvector/pgvector:0.8.0-pg16`	5432	bases `aptiso` et `aptiso_ai` (rôle dédié)
`redis`	`redis:7.4-alpine`	6379	file des traitements IA
`ai` (profil `ai`)	construite depuis `apps/ai`	127.0.0.1:8000	API du service d'IA
`ai-worker` (profil `ai`)	même image	—	worker arq

- **Volumes nommés** : les données survivent à la recréation des conteneurs.
- **Contrôles de santé** (`pg_isready`, `redis-cli ping`) et `depends_on: condition: service_healthy` : le service d'IA ne démarre qu'une fois la base et Redis prêts.
- **`restart: unless-stopped`** : un conteneur arrêté est relancé et le worker se reconnecte à Redis.
- **Profils** : `docker compose up -d` lance PostgreSQL et Redis ; le profil `ai` ajoute le service d'IA et son worker.
- **Image du service d'IA** : base `python:3.12-slim`, dépendances figées, utilisateur non root, contrôle de santé sur `/v1/health/live` ; au démarrage, `alembic upgrade head` puis `uvicorn`.

8.2 Diagramme de déploiement

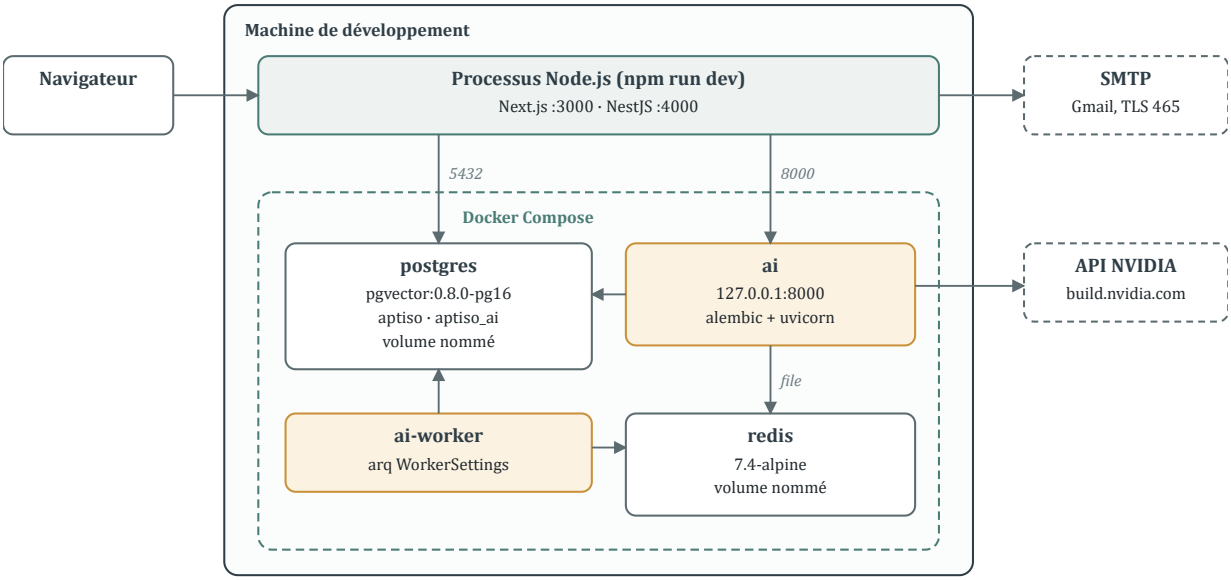


Figure 8.1 : Diagramme de déploiement de l'environnement de développement.

Dans le réseau Docker, le service d'IA joint PostgreSQL et Redis par leur nom de service (`AI_DB_HOST=postgres`, `REDIS_URL=redis://redis:6379`) ; seul son port d'API est publié, et uniquement sur l'interface locale.

8.3 Configuration et secrets

La configuration passe par des variables d'environnement, chargées depuis des fichiers `.env` exclus de Git : URL des bases, secrets JWT, paramètres SMTP, jeton de service et URL du service d'IA, clé de chiffrement des connecteurs, identifiants OAuth Google, clé de l'API NVIDIA et noms des modèles. Un fichier `.env.example` documente chaque variable sans valeur sensible.

8.4 Démarrage de l'environnement

```
docker compose --profile ai up -d --build    # PostgreSQL, Redis, service IA, worker
cd apps/backend && npx prisma migrate deploy  # schéma de la base métier
cd ../.. && npm run dev                       # frontend :3000 + backend :4000
```

Chapitre 9

Tests et qualité

9.1 Stratégie : tester avec les vrais composants

Les tests utilisent une vraie base PostgreSQL, le vrai service d'IA et les vrais modèles plutôt que des simulations : un test doit échouer quand l'application échouerait pour un utilisateur. Le scénario de bout en bout appelle les services du backend comme le feraient les contrôleurs, dans une base dédiée (`aptiso_test`) migrée avant chaque exécution ; seul l'envoi réel des e-mails est intercepté, pour vérifier ce qui serait parti.

9.2 Suites de tests

Tableau 9.1 : Suites de tests et contrôles de qualité.

Suite	Commande	Contenu
Scénario de bout en bout	`npm run test:scenario`	**202 vérifications** : un projet complet à travers les cinq phases, les règles d'audit, les tâches, notifications, demandes, preuves, exports et le miroir Drive simulé
Contrôles IA réels	`npm run test:ai`	constructeur de contexte, joignabilité du service avec son jeton, refus d'un mauvais jeton, erreur 503 quand le service est injoignable
Évaluation de l'IA	`node test/ai-eval.js`	28 cas annotés (assistant et revue), notés sans modèle ; tableau de résultats et comparaison entre versions d'invites (section 6.9)
Service d'IA	`pytest` (`apps/ai`)	santé de l'API, de la base, de pgvector et de Redis
Typage et style	`npm run typecheck`, `npm run lint`	`tsc --noEmit` et ESLint sur le frontend et le backend

Chaque évolution passe le typage, le lint et le scénario ; une évolution de l'IA est en plus essayée réellement, par exemple l'assistant interrogé sur le projet de démonstration et la vérification qu'un autre membre ne peut pas lire une conversation privée.

9.3 Données de démonstration

Deux scripts (`test/novateck.seed.js` et `test/novateck.finish.js`) construisent une entreprise fictive réaliste, *NovaTeck* (140 salariés, plateforme SaaS de paie), en passant par les **services réels** de l'application, si bien que toutes les règles métier s'appliquent : 10 comptes couvrant les sept rôles ISO, 8

exigences légales et contractuelles, 15 actifs et 21 risques approuvés par leurs propriétaires, une SoA de 93 mesures, 21 politiques approuvées, des formations, objectifs, incidents, non-conformités, un audit interne et une première revue de direction, soit 39 étapes sur 40. Ces scripts ont révélé deux règles manquantes, corrigées depuis : l'ouverture préalable des objectifs par défaut et l'obligation de publier dans la bibliothèque le document d'une étape de registre.

Chapitre 10

Limites et perspectives

10.1 Limites

- **Corpus d'évaluation encore restreint.** Les 28 cas annotés couvrent les fonctions principales sur un seul projet de démonstration ; ils ne suffisent pas à garantir la qualité sur des organisations très différentes ni pour la lecture de preuves numérisées.
- **Dépendance à un fournisseur de modèles externe.** Les extraits envoyés aux modèles quittent l'infrastructure de l'organisation, et l'API impose des quotas et une latence variable. Le modèle de secours et les comportements sans IA limitent l'impact d'une panne, sans supprimer la dépendance.
- **Validation en environnement de développement uniquement.** L'application n'a pas encore été déployée en production ni soumise à des tests de charge.

10.2 Perspectives

- **Texte officiel de la norme sous licence** : permettre à une organisation qui détient sa propre licence de téléverser sa copie, indexée et visible de ses seuls projets, par le mécanisme de connaissance partagée déjà en place.
- **Évaluation continue** : élargir le jeu de cas (autres secteurs, preuves numérisées, assistants de rédaction), l'exécuter automatiquement à chaque changement d'invite dans la chaîne d'intégration, et afficher les statistiques de `/v1/metrics` dans un tableau de bord.
- **Industrialisation** : images Docker du frontend et du backend, pipeline CI/CD (typage, lint, scénario), HTTPS, sauvegardes et supervision.
- **Stockage objet des preuves** (S3 ou MinIO) à la place du disque local, pour la redondance et la montée en charge.
- **Modèles hébergés par l'organisation** (modèles ouverts sur ses propres serveurs) pour les clients qui exigent que les données ne quittent pas leur infrastructure ; le registre de capacités le permet sans modifier les fonctionnalités.
- **Tests d'interface automatisés** (Playwright) et utilisation effective du paquet `@aptiso/shared-types` par les deux applications.
- **Autres référentiels** (ISO 9001, ISO/IEC 27701) sur le même moteur de phases, grâce au modèle `ComplianceFramework`, et interface bilingue français / anglais.

Conclusion générale

APTISO couvre le cycle complet d'un projet ISO/IEC 27001, de la préparation à la maintenance après certification. Son architecture sépare clairement trois responsabilités : un frontend sans logique métier, un backend qui détient les données et applique les règles de la norme, et un service d'IA isolé qui ne voit que ce que la personne a le droit de voir.

Le principe directeur du projet est que l'IA est utile lorsqu'elle est encadrée par une ingénierie déterministe : le modèle lit, résume et propose ; le code valide, trace et refuse ce qui n'est pas fondé ; la personne décide. Ce principe se retrouve dans la validation des sorties structurées, la vérification des citations, la journalisation de chaque appel et l'existence d'un repli sans IA pour chaque fonction. Il est désormais mesuré : un jeu de cas annotés montre que l'assistant ne fabrique aucune information sur les cas évalués et que la revue détecte tous les défauts introduits, et une base de 118 guides rédigés pour APTISO lui permet d'enseigner la norme sans en reproduire le texte.

Le projet de démonstration NovaTeck, construit par les services réels et vérifié par un scénario de 202 contrôles, montre que la plateforme tient sur un cas réaliste. Les prochaines étapes portent sur l'enrichissement de la connaissance de la norme, la mesure de la qualité de l'IA et l'industrialisation du déploiement.

Bibliographie

[1] ISO/IEC. *ISO/IEC 27001:2022 — Sécurité de l'information, cybersécurité et protection de la vie privée — Systèmes de management de la sécurité de l'information — Exigences*. ISO, 2022.

[2] ISO/IEC. *ISO/IEC 27002:2022 — Mesures de sécurité de l'information*. ISO, 2022.

[3] Advisera. *Conformio — ISO 27001 compliance software*. https://advisera.com/conformio/

[4] Vercel. *Next.js Documentation*. https://nextjs.org/docs

[5] NestJS. *NestJS Documentation*. https://docs.nestjs.com

[6] Prisma. *Prisma ORM Documentation*. https://www.prisma.io/docs

[7] S. Ramírez. *FastAPI Documentation*. https://fastapi.tiangolo.com

[8] pgvector. *Open-source vector similarity search for Postgres*. https://github.com/pgvector/pgvector

[9] G. V. Cormack, C. L. A. Clarke, S. Büttcher. *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods*. SIGIR, 2009.

[10] NVIDIA. *NVIDIA API Catalog*. https://build.nvidia.com

[11] S. Colvin. *arq — job queues and RPC in Python with asyncio and Redis*. https://arq-docs.helpmanual.io

[12] D. M'Raihi et al. *RFC 6238 — TOTP: Time-Based One-Time Password Algorithm*. IETF, 2011.

[13] OWASP Foundation. *OWASP Top 10 et Application Security Verification Standard*. https://owasp.org

[14] Pydantic. *Data validation using Python type hints*. https://docs.pydantic.dev

[15] Tiptap. *Tiptap Editor Documentation*. https://tiptap.dev

[16] tldraw. *tldraw SDK Documentation*. https://tldraw.dev

[17] Docker. *Docker Compose Documentation*. https://docs.docker.com/compose/

[18] OpenAPI Initiative. *OpenAPI Specification 3*. https://spec.openapis.org/oas/latest.html ; NestJS, *OpenAPI (Swagger)*. https://docs.nestjs.com/openapi/introduction

[19] A. Bayoudh. *APTISO — code source*. https://github.com/aminaoamina/APTISO